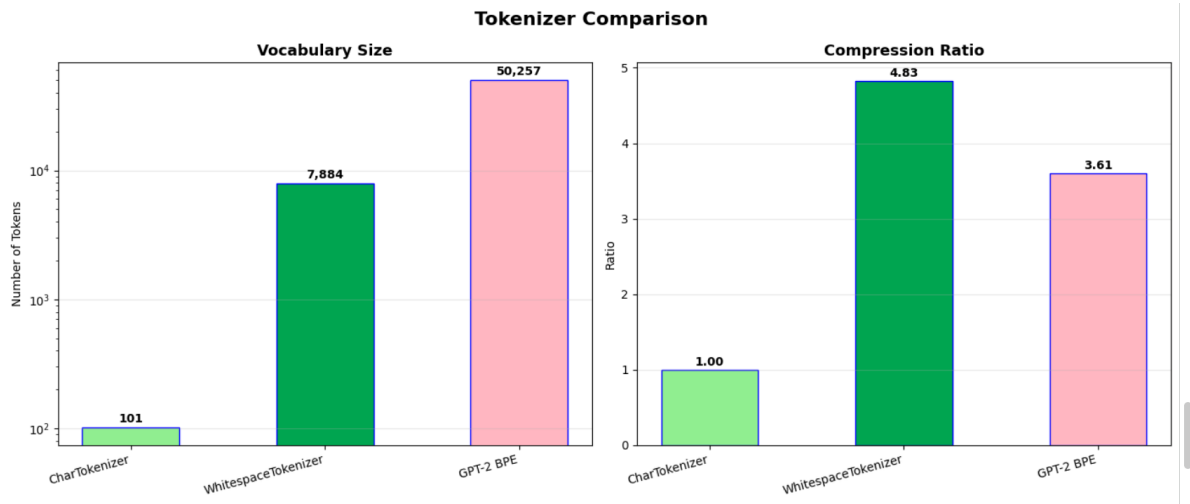
# Section 1: Tokenization Trade-offs

1)**Compute: compression ratio (characters / tokens), vocabulary size, and average token length**




**2)Explain the trade-off between vocabulary size and sequence length. Why might a small vocabulary (charlevel) lead to worse model performance despite preserving all information?**

- Normally, if vocabulary size increases sequence length will be decreasing and the viceversa which could be visualized below. And the sweetspot is with tiktoken

==========

| METRIC | CHAR | WHITESPACE | TIKTOKEN |
|---|---|---|---|
| Total Characters | 763,083 | | |
| Total Tokens | 763,083 | **158,**088 | 211,566 |
| Vocab Size | 101 | **7,**884 | 50,257 |
| Compression Ratio | 1.00 | 4.83 | 3.61 |
| Avg Token Length | 1.00 | 3.84 | 3.63 |

==========

Small vocabulary lead to worse model performance because

- there will be long sequences which ultimately lead to memory explosion
- there will be long dependency problems as understanding a single sentence might require additional traversing
- There will be need of many steps to understand semantic meaning unlike tiktoken which understands meaning via single token
- Transformer attention = $O(n^2)$ ie more computation

# Section 2: Model Parameters : Inference Memory Architecture

1)

| Component | Formula | Small (124M) | Medium (355M) | Large (774M) |
|---|---|---|---|---|
| **Embeddings** | | | | |
| Token | $V \times d_{\text{model}}$ | - | - | - |
| Positional | $T \times d_{\text{model}}$ | - | - | - |
| **Per Layer** | | ×12 | ×24 | ×36 |
| Attention QKV | $3 \times d_{\text{model}}^2$ | - | - | - |
| Attention Out | $d_{\text{model}}^2$ | - | - | - |
| MLP Up | $d_{\text{model}} \times d_{\text{ff}}$ | - | - | - |
| MLP Down | $d_{\text{ff}} \times d_{\text{model}}$ | - | - | - |
| LayerNorms (2 LNs) | $4 \times d_{\text{model}}$ | - | - | - |
| **Total Per Layer** | | ~7.1M | ~12.6M | ~19.7M |
| **All Layers** | - | - | - | - |
| **TOTAL PARAMS** | - | - | - | - |
| **FP32 Memory** | - | - | - | - |
| **FP16 Memory** | - | - | - | - |

```
================================================================================
PARAMETER CENSUS
================================================================================
Component       Formula              Small (124M)  Medium (355M)   Large (774M)
--------------------------------------------------------------------------------
Token Embed     V × d_model            38,597,376     51,463,168     64,328,960
Pos Embed       T × d_model               786,432      1,048,576      1,310,720
--------------------------------------------------------------------------------
Attn QKV        3 × d_model²            1,771,776      3,148,800      4,919,040
Attn Out        d_model²                  590,592      1,049,600      1,639,680
MLP Up          d_model × dff           2,362,368      4,198,400      6,558,720
MLP Down        dff × d_model           2,360,064      4,195,328      6,554,880
LayerNorms      4 × d_model                 3,072          4,096          5,120
--------------------------------------------------------------------------------
Total Per Layer Sum above                    ~7.1M         ~12.6M         ~19.7M
All Layers      Per Layer × L          85,054,464    302,309,376    708,387,840
--------------------------------------------------------------------------------
TOTAL           Embed + Layers        124,439,808    354,823,168    774,030,080
--------------------------------------------------------------------------------
FP32_MB         total×4 (MB)               474.70        1353.54        2952.69
FP16_MB         total×2 (MB)               237.35         676.77        1476.35
```

```
================================================================================
TASK 2: ACTIVATION MEMORY ANALYSIS
(Inference | batch_size=1 | seq_length=1024)
================================================================================
```

| Component | Formula | Small | Medium | Large |
|---|---|---|---|---|
| Input Embed | B × T × d_model | 786,432 | 1,048,576 | 1,310,720 |
| QKV Proj | B × 3 × T × d_model | 2,359,296 | 3,145,728 | 3,932,160 |
| Attn Scores | B × H × T × T | 12,582,912 | 16,777,216 | 20,971,520 |
| MLP Act | B × T × dff | 3,145,728 | 4,194,304 | 5,242,880 |
| LN Input | B × T × d_model | 786,432 | 1,048,576 | 1,310,720 |
| Peak Floats | max(Attn,MLP)+LN | 13,369,344 | 17,825,792 | 22,282,240 |
| FP32_MB | Peak × 4 (MB) | 51.00 | 68.00 | 85.00 |
| FP16_MB | Peak × 2 (MB) | 25.50 | 34.00 | 42.50 |

**2)Calculate: Peak activation memory for the Small model (in MB) Ratio of activation memory to parameter memory for each model size At what model size do activations become negligible compared to parameters?**

**Peak Activation Memory for Small model**

Peak Floats = max(attn_scores, mlp_act) + layernorm_inp attn_scores

= 1 × 12 × 1024 × 1024 = 12,582,912

mlp_act = 1 × 1024 × 3072 = 3,145,728

layernorm_inp(residual)= 1 × 1024 × 768 = 786,432

peak = max(12,582,912 , 3,145,728) + 786,432 = 12,582,912 + 786,432 = 13,369,344 floats

FP32_MB = 13,369,344 × 4 / $1024^2$ = 51.00 M

**Ratio of activation memory to parameter memory for each model size**

**Small model**

Activation memory = 51.00 MB

Parameters memory = 474.70 MB

Ratio = 51.00 / 474.70 = 0.1074 = 10.74%

**Medium model**

Activation memory = 68.00 MB

Parameters memory = 1353.54 MB

Ratio = 68.00 / 1353.54 = 0.0502 = 5.02%

**Large Model:**

Activation Memory = 85.00 MB

Parameters Memory = 2952.69 MB

Ratio = 85.00 / 2952.69 = 0.0288 = 2.88%

**At what model size do activations become negligible compared to parameters?**

Small = 51.00 / 474.70 = 0.1074

Medium = 68.00 / 1353.54 = 0.0502

Large = 85.00 / 2952.69 = 0.0288

As per the calculations at nearly 0.028 , activations become negligible

**Task 3: Scaling Laws**

**1)Analysis (200 words): "Why does the Medium model use ~2.9× the memory of Small, but Large only uses ~2.2× the memory of Medium, despite both adding 12 layers?" (Hint: embedding matrices are constant size; only transformer blocks scale with L.)**

As mentioned embedding matrices are constant sizes ie fixed for all models but cost is getting diluted if we keep adding more layers as the total memory base keeps growing. (ratio changes in accordance with base)

Let us assume below example

Memory = E_fixed + L × B_per_layer

| Model | Layers | Embedding | Layer Cost | Total |
|---|---|---|---|---|
| Small | 6 | 100MB | 6 × 10 = 60MB | 160MB |
| Medium | 18 | 100MB | 18 × 10 = 180MB | 280MB |
| Large | 30 | 100MB | 30 × 10 = 300MB | 400MB |

Ratios:

- Medium ÷ Small = 280 ÷ 160 = 2.9×
- Large ÷ Medium = 400 ÷ 280 = 2.2×

For the small model. Total base is 160 MB fixed embedding cost (100MB) contributes **62%** of total memory, making the ratio large **(2.9×) and** When going from Medium → Large, the base

(280MB) is larger, so the fixed embedding cost (100MB) only contributes 35% of total memory, making the ratio smaller (2.2×**).** As layers increase, the fixed embedding cost gets "diluted" across a larger total, so the relative impact decreases — causing the ratio to drop despite adding the same 12 layers each time**.**

**2)Activation memory calculation showing the O(T 2 ) attention score block dominates ephemeral memory**

```
================================================================================
O(T²) ATTENTION ANALYSIS
================================================================================

Tensor         Complexity                                          Floats
──────────────────────────────────────────────────────────────────────────
Input Embed    B × T × d        → O(T)                            786,432
QKV Proj       B × 3 × T × d    → O(T)                          2,359,296
Attn Scores    B × H × T × T    → O(T²)                        12,582,912
MLP Act        B × T × dff      → O(T)                          3,145,728
LN Input       B × T × d        → O(T)                            786,432
──────────────────────────────────────────────────────────────────────────

Peak WITH Attention            :     13,369,344 floats
Peak WITHOUT Attention         :      3,932,160 floats
Attention Overhead             : 3.40× more memory!

T              Attn Scores  Ratio vs T=1024
──────────────────────────────────────────
T=512            3,145,728             0.2×
T=1024          12,582,912             1.0×
T=2048          50,331,648             4.0×
T=4096         201,326,592            16.0×
```

Here, Input Embeddings, QKV Projections, MLP Activations and Layer Normalization are using O(T) ie it is using memory less compared to attention scores

Attention score = B × H × T × T. (Doubling T causes attention memory to **quadruple)**

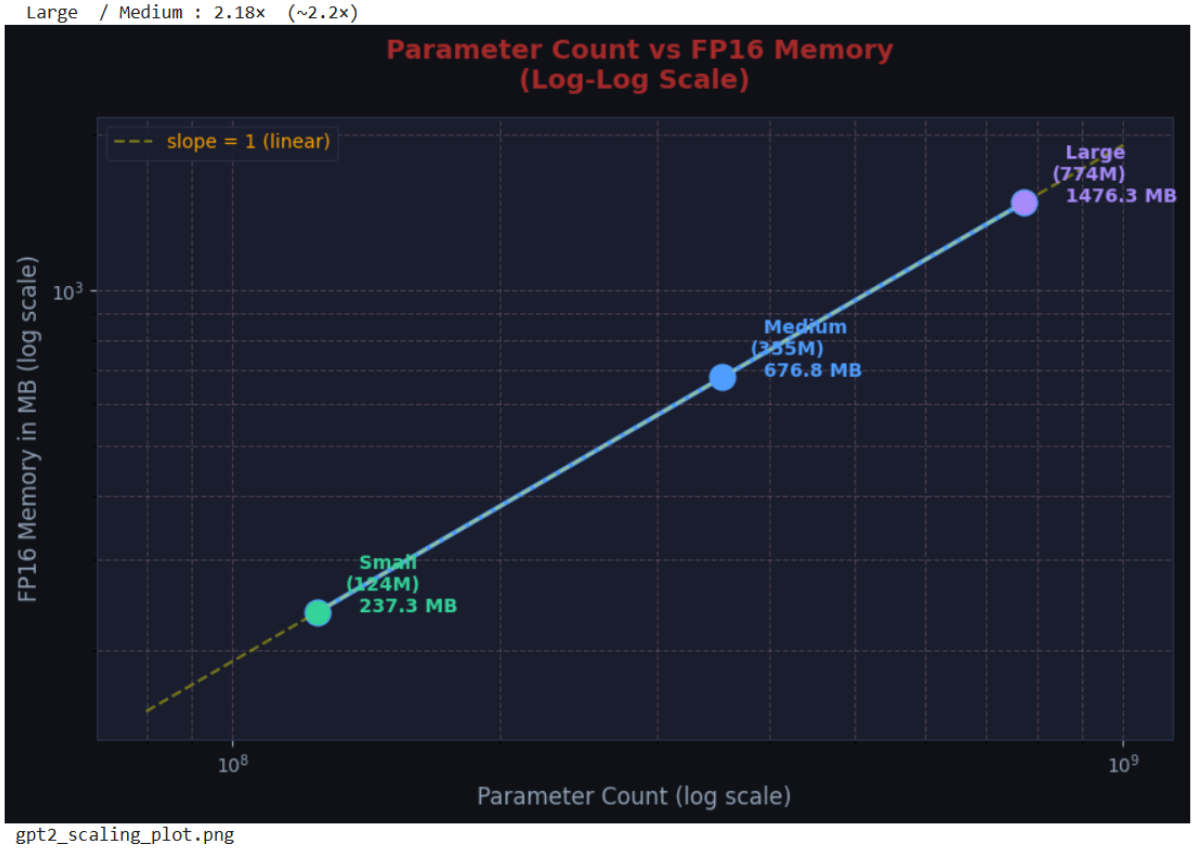
Which holds a highest memory as every token has to check with every other token, making it $O(T^2)$ and from snip we could see attention alone contributes 12,582,912 while remaining all constitute a sum of 3,932,160 floats.

| Sequence Length | Attention Memory | Ratio |
|---|---|---|
| T=512 | 3,145,728 | 0.25× |
| T=1024 | 12,582,912 | 1.0× |
| T=2048 | 50,331,648 | 4× |
| T=4096 | 201,326,592 | 16× |

Sequence Length Attention Memory Ratio

**3)Plot parameter count vs. FP16 memory for all three models on a log-log scale**

```
Medium / Small  : 2.85×  (~2.9×)
Large  / Medium : 2.18×  (~2.2×)
```




gpt2_scaling_plot.png

4) **Verify your parameter counts match the reference values (±0.1%): Small: 124,439,808 params Medium: 354,823,168 params Large: 773,903,616 params**

==========

VERIFICATION

==========

Small  : Calculated =  124,439,808 | 0.0000% diff | Matched

Medium  : Calculated =  354,823,168 | 0.0000% diff | Matched

Large  : Calculated =  774,030,080 | 0.0163% diff | Matched

# Section 3: From CPU to GPU : Inference Profiling & Optimization (16 points)

## Pre flight check

**Section 3: From CPU to GPU : Inference Profiling & Optimization**

```python
#Pre Flight check
import torch
print(f"PyTorch: {torch.__version__}")
print(f"CUDA Available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
  print(f"Device: {torch.cuda.get_device_name(0)}")
  print(f"VRAM:{torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
!nvidia-smi --query-gpu=name,memory.total,memory.free --format=csv
```

```
PyTorch: 2.10.0+cu128
CUDA Available: True
Device: Tesla T4
VRAM:15.64 GB
name, memory.total [MiB], memory.free [MiB]
Tesla T4, 15360 MiB, 14910 MiB
```

Start coding or generate with AI.

**1)Metrics to collect: Cold-start latency (time to first token) Steady-state latency (average latency over tokens 10–50) Throughput (tokens/sec) System RAM usage (psutil or torch.cuda.memory_allocated())**

**TASK 1: CPU BASELINE RESULTS**

**==========**

| Model | First Token | Steady State | Throughput | RAM |
|---|---|---|---|---|
| | | | | |
| Small 124M | 437.6ms | 497.1ms | 1.48t/s | 5.2MB |
| Medium 355M | 1138.4ms | 1637.1ms | 0.50t/s | 6.2MB |
| Large 774M | 1667.4ms | 3183.2ms | 0.25t/s | 13.2MB |

**Task 2: GPU Migration & Basic Acceleration**

**1)Device transfer time: Time moving parameters from CPU RAM -> GPU VRAM (use time.time() before and after .cuda()) Inference throughput: Same task as Task 1; use torch.cuda.synchronize() before and after generation Peak VRAM usage: torch.cuda.max_memory_allocated() after each run**

```
==========================================================================================
TASK 2: GPU BENCHMARK RESULTS
==========================================================================================
Model            Transfer  First Tok     Steady   FP32 t/s   FP16 t/s  VRAM FP32  VRAM FP16  Spdup FP32  Spdup FP16
------------------------------------------------------------------------------------------
Small 124M        528.7ms     11.8ms     11.1ms    88.69t/s    69.22t/s    591.3MB    318.7MB      59.9x      46.8x
Medium 355M       613.7ms     40.4ms     24.1ms    33.42t/s    52.76t/s   1523.4MB    788.4MB      66.8x     105.5x
Large 774M       7827.7ms     49.0ms     51.3ms    14.84t/s    38.29t/s   3265.9MB   1633.6MB      59.4x     153.1x
==========================================================================================
```

**2) Compute the speedup ratio (CPU time / GPU time) for each model size.**

**CPU Throughput (**

| |
|---|
| Small = 1.48 t/s |
| Medium = 0.50 t/s |
| Large = 0.25 t/s |

**GPU Throughput**

| |
|---|
| Small = 88.69 t/s (FP32)  69.22 t/s (FP16) |
| Medium = 33.42 t/s (FP32)  52.76 t/s (FP16) |
| Large = 14.84 t/s (FP32)  38.29 t/s (FP16) |

**Speedup = GPU t/s ÷ CPU t/s**

**FP32 Speedup:**

| |
|---|
| Small = 88.69 ÷ 1.48 = 59.9× |
| Medium = 33.42 ÷ 0.50 = 66.8× |
| Large = 14.84 ÷ 0.25 = 59.4× |

**FP16 Speedup:**

| |
|---|
| Small = 69.22 ÷ 1.48 = 46.8× |
| Medium = 52.76 ÷ 0.50 = 105.5× |
| Large = 38.29 ÷ 0.25 = 153.1× |

**3)Think about this: Does speedup scale linearly with model size? Why does the Large model hit ~20–30× while Small only manages ~5–10×? (Hint: fixed GPU overhead doesn't go away for small models.)**

- No, speedup does not scale linearly with model size because gpu overhead remains constant irrespective of model size. The hit difference is due to large models have more matrix operations that fully utilize GPU **FP16 tensor cores**.
- For small models, this overhead dominates GPU time, limiting speedup. For large models, actual computation outweighs the overhead, allowing the GPU to run at full capacity — resulting in significantly higher speedup.

## Task 3: Optimization Experiments (4 points)

**Option A: Mixed Precision (FP16) Measure speedup vs. FP32 (expect ~1.5–2× on T4 due to Tensor Cores)**

| Config | Throughput | Latency | VRAM(MB) | Perplexity |
|---|---|---|---|---|
| FP32 | 24.65t/s | 40.57ms | 3976.0 | 10.3583 |
| FP16 | 54.93t/s | 18.21ms | 3230.4 | 10.3531 |
| Speedup | 2.23× | | | |

**Check output quality: compute perplexity on a test sentence and confirm it stays within 1% of FP32**

FP16 Throughput : 54.93 tok/s

FP16 Latency : 18.21 ms/tok

FP16 VRAM : 3230.4 MB

FP16 Perplexity : 10.3531

Speedup : 2.23× [Formula: 54.93 / 24.65]

VRAM Saved : 745.6 MB [Formula: 3976.0 - 3230.4]

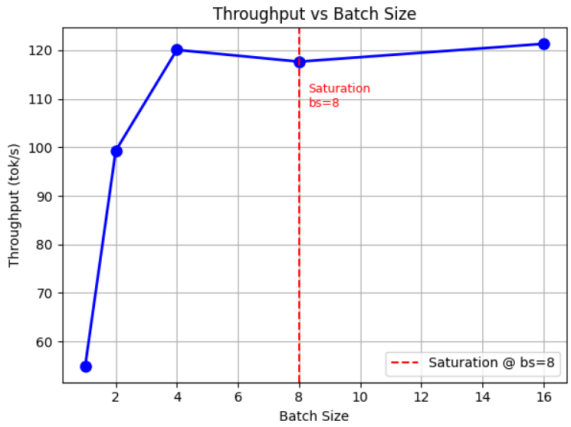
PPL Diff : 0.0501% [Formula: |10.3531 - 10.3583| / 10.3583 * 100] within 1%

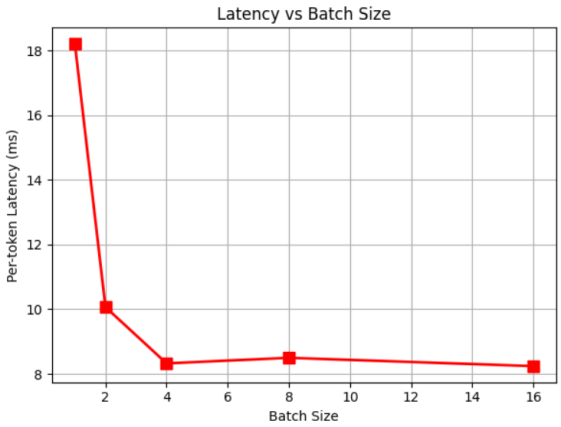
**Option B: Batch Size Scaling**

**1) Test batch sizes: [1, 2, 4, 8, 16] (**

| Batch Size | Throughput | Latency | VRAM |
|---|---|---|---|
| **bs=1** | **54.91 tok/s** | **18.21 ms** | **3084.4 MB** |
| **bs=2** | **99.33 tok/s** | **10.07 ms** | **3154.9 MB** |
| **bs=4** | **120.10 tok/s** | **8.33 ms** | **3307.7 MB** |
| **bs=8** | **117.67 tok/s** | **8.50 ms** | **3577.1 MB** |
| **bs=16** | **121.34 tok/s** | **8.24 ms** | **4141.5 MB** |

**2)Plot throughput (tokens/sec) vs. batch size**




**3)Mark the saturation point where adding more to the batch stops helping**

| **Batch Size** | **Throughput (tok/s)** | **Latency (ms/tok)** | **VRAM (MB)** | **Gain %** |
|---|---|---|---|---|
| **bs=1** | **54.91** | **18.21** | **3084.4** | **—** |
| **bs=2** | **99.33** | **10.07** | **3154.9** | **+80.9%** |
| **bs=4** | **120.10** | **8.33** | **3307.7** | **+20.9%** |
| **bs=8** | **117.67** | **8.50** | **3577.1** | **-2.0%** |
| **bs=16** | **121.34** | **8.24** | **4141.5** | **+3.1%** |

Saturation Point: The saturation point was reached at bs=8, where throughput gain dropped to -2.0% (below the 10% threshold) relative to bs=4, despite VRAM increasing by 269 MB. Beyond bs=4, adding more batches stops helping — GPU cores are fully utilized and throughput plateaus between 117–121 tok/s.

**4)Compute per-token latency at each batch size and discuss the latency/throughput trade-of**

| Batch Size | Throughput (tok/s) | Latency (ms/tok) | VRAM (MB) |
|---|---|---|---|
| bs=1 | 54.91 | 18.21 | 3084.4 |
| bs=2 | 99.33 | 10.07 | 3154.9 |
| bs=4 | 120.10 | 8.33 | 3307.7 |
| bs=8 | 117.67 | 8.50 | 3577.1 |
| bs=16 | 121.34 | 8.21 | 4141.5 |

As batch size increases from bs=1 to bs=4, latency improves significantly from 18.21 ms → 8.33 ms (2.2× lower), while throughput nearly triples from 54.91 → 120.10 tok/s. This shows the GPU is being progressively filled with useful parallel work, amortizing the fixed memory-load cost across more sequences.

**5)Observation: "At batch size X, we hit approximately Y% of the T4's theoretical peak FLOPs"**

T4 FP16 Peak = 65 TFLOPS GPT2-Medium Params = 355M FLOPs per token = 2 × 355M = 710M FLOPs

= (120.10 × 710,000,000) / 65,000,000,000,000 × 100 = 85,271,000,000 / 65,000,000,000,000 × 100 = 0.13%

At batch size 4, we hit approximately 0.13% of the T4's theoretical peak FLOPs (65 TFLOPS), confirming that autoregressive inference is memory-bandwidth bound rather than compute-bound.

**FINAL COMPARISON MATRIX**

| Configuration | Device | Precision | Batch | Tok/s | VRAM (GB) | Speedup vs CPU |
|---|---|---|---|---|---|---|
| Small 124M | CPU | FP32 | 1 | 1.48 | N/A | 1.0× |
| Small 124M | T4 | FP32 | 1 | 88.69 | 0.577 | 59.9× |
| Small 124M | T4 | FP16 | 1 | 69.22 | 0.311 | 46.8× |
| Medium 355M | T4 | FP16 | 8 | 117.67 | 3.493 | 235.3× |
| Large 774M | T4 | FP16 | 4 | 51.88 | 3.388 | 207.5× |

**Submission Checklist:**

**Took assistance from Claude for some codes and reports**